SCALER School of Technology · Priyansh Saxena

The ML System Design Framework

Business Objectives ▪ Four Requirements ▪ Iterative Process ▪ Problem Framing

Priyansh Saxena

Scaler School of Technology | ML System Design and LLMOps

Week 1 — Session 2/2

ROADMAP

Agenda

1. From ML Metrics to Business Metrics
2. Four Requirements for ML Systems
3. The Iterative Development Process
4. Framing ML Problems
5. Mind vs. Data

TOC

1

From ML Metrics to Business Metrics

1 Why ML Projects Get Killed

The pattern Chip Huyen observes

- Data scientists optimise ML metrics (loss, AUC, F1)
- Business stakeholders track revenue, retention, cost
- The two groups talk past each other
- Project is cancelled: "we spent 6 months and nothing changed"

The key insight

"Most companies don't care about ML metrics unless they move **business metrics**."
— Chip Huyen, *Designing ML Systems*, Ch. 2

The right question to ask first

- What **business metric** should improve?
- By how much, in what timeframe?
- How does the ML metric *connect* to it?
- What is the cost of being wrong?

Rule of thumb

Every ML metric you optimise should have a clear, traceable path to a business outcome. If you cannot draw that path, reconsider whether ML is the right solution.

1 Translating Business Metrics to ML Metrics

Business metric	What it measures	Proxy ML metric
Monthly Active Users	Engagement and product stickiness	Predicted session length, click-through rate
Subscription cancellation rate	Long-term retention	Predicted churn probability
Revenue per user	Monetisation effectiveness	Predicted purchase probability, basket size
Customer support cost	Operational efficiency	Auto-resolution rate, ticket routing accuracy
Fraud losses ($)	Risk exposure	Precision/recall of fraud classifier at fixed threshold

The proxy ML metric should move in the same direction as the business metric — always verify this empirically.

1 Netflix Take-Rate: A Worked Example

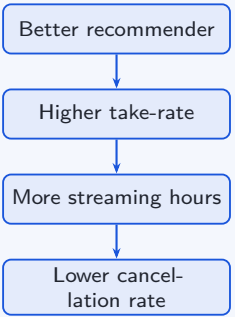
The business question

- Netflix runs a recommender for billions of impressions/day
- Which ML metric should the team optimise?
- Raw accuracy on held-out ratings? NDCG?

Netflix's answer: take-rate

- take-rate $= \dfrac{\#\text{ quality plays}}{\#\text{ recommendations shown}}$
- Higher take-rate $\Rightarrow$ better recommender
- Higher take-rate $\Rightarrow$ more streaming hours
- More streaming hours $\Rightarrow$ lower cancellation rate

The causal chain



Source: Chip Huyen, Designing ML Systems, Ch. 2

1 “Magically: Possible. Overnight: No.”

The deployment maturity curve

- ROI on ML depends heavily on organisational maturity
- Among companies with models in production **5+ years**:
  - 75% can deploy a new model in **under 30 days**
- Among companies **just getting started**:
  - 60% take **over 30 days** to deploy their first model

Implication

If you are at a company with no ML infrastructure, your first project will take longer than you expect. Plan for it — don’t promise “overnight magic.”

What maturity unlocks

- Reusable data pipelines (no re-ingestion from scratch)
- Shared feature store (no feature recomputation)
- Automated training pipeline (no manual runs)
- One-click deployment via model registry
- Monitoring already wired up

Practical advice

Invest in infrastructure early. Each project after the first gets faster — but only if you build the foundations right.

2

Four Requirements for ML Systems

2 Reliability: When ML Fails Silently

Traditional software failures

- Crash with a stack trace
- 500 error code to the user
- Clear signal: something broke
- On-call engineer gets paged immediately

ML system failures

- Model returns a prediction — confidently
- No exception, no error log
- Prediction is just *wrong*
- Nobody notices for weeks

The Google Translate problem

"If you use Google Translate to translate a sentence into a language you don't know, it might be very hard for you to tell even if the translation is wrong."
— Chip Huyen, *Designing ML Systems*, Ch. 2

What reliability requires

- Monitoring prediction distributions, not just uptime
- Proxy metrics that correlate with output quality
- Fallback strategies (rule-based backup, previous model version)

2 Scalability: 8,000 Models for 8,000 Customers

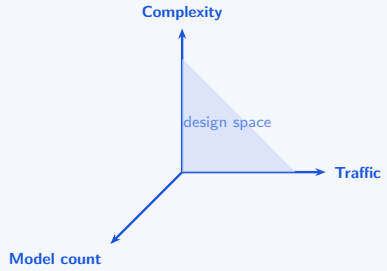
Scalability is not just traffic

- Complexity: one model vs. thousands of models
- Traffic: 100 requests/day vs. 100 million
- Model count: a startup Chip Huyen worked with had **8,000 models** — one per enterprise customer
- Each model needs its own: training data, retraining schedule, monitoring, deployment

Amazon Prime Day failure

Amazon's autoscaling failed during Prime Day.
Cost: **$72–99M in a single hour**.
Scalability is not a nice-to-have.

Three axes of scale



2 Maintainability and Adaptability

Maintainability

- Multiple contributors must work on the same system: ML engineers, DevOps, subject-matter experts
- Each group has different tools, languages, and priorities
- Requirements:
  - Code is **documented** and reviewable
  - Data is **versioned** (not just "latest.csv")
  - Models are **reproducible**: same code + same data = same model
- Without maintainability, you cannot onboard new engineers or debug production issues

Adaptability

- Data distributions shift over time (users change behaviour)
- Business requirements change (new markets, regulations)
- The system must **adapt without service interruption**
- This is different from reliability — you need both

What adaptability looks like

- Automated retraining pipelines
- Canary deployments for new models
- Feature flag infrastructure for quick rollbacks
- Regular model evaluation cadences

2 The Four Requirements: Summary

Requirement	What it means	How it fails without it
Reliability	System performs correctly even in the face of adversity: bad inputs, infrastructure failures, edge cases	Predictions degrade silently; users and engineers have no idea until business metrics drop
Scalability	Handles growth in traffic, model complexity, and number of models without architectural rework	System breaks under load; Prime Day failures; 8K models become unmanageable
Maintainability	Multiple teams can understand, modify, and extend the system independently	Only the original author can change anything; onboarding takes months; bugs are impossible to trace
Adaptability	Can evolve with shifting data distributions and business requirements without service interruption	Model becomes stale; adapting requires full system rebuild; business loses competitive edge

5-Minute Break

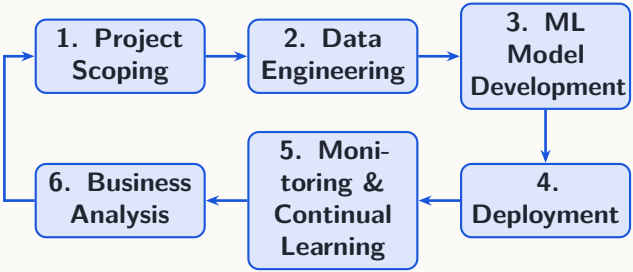
Your model is reliable, scalable, maintainable, and adaptable.

Congratulations — it also hasn't been deployed yet. Grab some water.

3

The Iterative Development Process

3 The 6-Step Development Cycle



- "Developing an ML system is an iterative and, in most cases, **never-ending process**." — Chip Huyen, Ch. 2
- The arrows do not only go forward — business analysis often sends you back to scoping
- Monitoring findings routinely trigger new data engineering work

3 What Each Step Really Involves

1. Project Scoping

- Define business objective and ML objective
- Identify stakeholders and constraints
- Estimate data availability and labelling cost
- Define success criteria *before* any modelling

2. Data Engineering

- Data collection, cleaning, and labelling pipelines
- Feature engineering and feature store population
- Data versioning and lineage tracking

3. ML Model Development

- Baseline model (always start simple)
- Iterative experimentation with tracked metrics
- Evaluation against held-out data and business criteria

4. Deployment

- Package model with preprocessing pipeline
- Choose serving pattern (sync, async, edge)
- Shadow mode or canary rollout before full launch

5. Monitoring & Continual Learning

- Track input drift, output drift, business metrics
- Trigger retraining when thresholds breached
- Incorporate new labelled data continuously

6. Business Analysis

- Did the business metric move?
- Is the model worth the ongoing infrastructure cost?
- What should the next iteration focus on?

3 Quiz 1

Q1 — The Cycle Is Not Linear

A team finishes model development (step 3) and discovers that the training data is missing a critical signal: location at the time of purchase.

Which step do they go back to? What specifically do they do there? Could this have been caught earlier — and at which step?

Q2 — Business Analysis Feedback

A churn prediction model achieves 87% AUC. After deployment, the cancellation rate does not change.

Does this mean the model failed? Or does it mean something else? What does step 6 (business analysis) tell the team to do next?

3 Quiz 1 — Answers

A1 — The Cycle Is Not Linear

Step 2 — Data Engineering.

They must collect the missing location signal, add it to the feature store, re-run the labelling pipeline, and version the new dataset. This could have been caught at **Step 1 (Project Scoping)** — a proper data audit and feature requirement checklist would have identified location as critical before any modelling began.

A2 — Business Analysis Feedback

The model did not necessarily fail — the *intervention* may have failed.

87% AUC means the model correctly ranks churners, but a prediction alone does not reduce churn. Step 6 tells the team to ask: **(a)** Was the model's output actually used to trigger a retention action (discount, outreach)? **(b)** Was the action effective? **(c)** Is AUC the right metric — or should we be optimising for *actionable* churn risk at a precision that makes retention cost-effective?

4

Framing ML Problems

4 What Makes Something an ML Problem?

An ML problem has three parts

1. **Inputs** — what data does the model receive?
2. **Outputs** — what should the model predict or generate?
3. **Objective function** — how do we measure how good the output is?

A common mistake

"Slow customer support" is **not** an ML problem.
It is a **business problem**. You must frame it.

Framing customer support routing

Part	Answer
Business problem	Support is slow
ML task	Route requests to 4 depts
Input	Customer request text
Output	Department label
Objective	Minimise misrouting rate
Task type	Multiclass classification

The business problem becomes solvable once you have inputs, outputs, and an objective function.

4 Task Type Taxonomy

Regression

- Output is a continuous value
- Loss: RMSE, MAE
- Examples: house price, demand forecast, ETA

Binary classification

- Output is one of two classes: yes/no, fraud/legit
- Loss: logistic loss (binary cross-entropy)
- Examples: spam filter, churn prediction, fraud detection

Multiclass classification

- Output is one of N classes
- Loss: cross-entropy over N classes
- Rule of thumb: need $\sim$100,000 examples for 1,000 classes
- Example: product category, language detection

Multilabel classification

- Each input can belong to *multiple* classes simultaneously
- Hardest to handle correctly
- "The one type of task I've seen companies having the most problems with" — Chip Huyen
- Example: document tagging, multi-genre movie labels

4 Problem Framing Changes Everything

Problem: predict next app to open

Framing 1: Multiclass classification

- N classes = N installed apps
- Output: which app (class index)
- Problem: when a new app is installed, you must **retrain** the model
- Adding 1 app invalidates the whole model

Framing 2: Regression (scoring)

- Model scores each app 0–1: "likelihood of opening"
- Output: a real-valued relevance score
- New app added? Just score it at serving time
- **No retraining required**

Comparison

	Multiclass	Regression
New app	Retrain	No retrain
Output	Class ID	Score
Ranking	Argmax	Sort by score
Flexibility	Low	High

Key lesson

The same real-world task can be framed as different ML tasks. The framing determines **scalability, maintainability, and cost**.

4 Decoupling Objectives

Newsfeed ranking problem

- You want to: filter spam, filter NSFW, filter misinformation, rank by quality, rank by engagement
- Naive approach: one combined loss

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{quality}} + \beta \cdot \mathcal{L}_{\text{engagement}}$$

- Problem: every time you change α or β, you must **retrain the entire model**
- Tuning α/β is product decision, not ML decision

The decoupled approach

1. Train a **separate** spam model
2. Train a **separate** NSFW model
3. Train a **separate** quality model
4. Train a **separate** engagement model
5. Combine scores at serving time:
   $\text{rank} = \alpha \cdot \text{quality_score} + \beta \cdot \text{engagement_score}$

The payoff

Product managers can now tune α and β **without retraining** any model. ML engineers and PMs are decoupled.

4 Quiz 2

Q1 — Frame This Business Problem

An e-commerce company says: "Our customers abandon their shopping carts too often."

Frame this as an ML problem. State: (1) the ML task type, (2) the input(s), (3) the output, (4) the objective function you would use, and (5) one business metric this model should improve.

Q2 — Decoupling Objectives

A content moderation system must handle both: (a) detecting hate speech and (b) detecting misinformation. A junior engineer proposes training one model with a combined loss. A senior engineer disagrees.

What is the senior engineer's argument? When would separate models be better? When might one combined model be acceptable?

4 Quiz 2 — Answers

A1 — Frame This Business Problem

Task type: Binary classification (abandon / not abandon).

Inputs: Cart contents, user session history, time-on-site, device type, coupon availability.

Output: Probability of cart abandonment in the next 30 min.

Objective: Binary cross-entropy loss; optimise precision at a threshold where a triggered retention action (discount pop-up) is cost-effective.

Business metric: Cart conversion rate (completed checkouts / cart initiations).

A2 — Decoupling Objectives

Senior engineer's argument: A combined loss requires retraining both detectors every time business priorities shift (e.g. tightening hate speech thresholds). Separate models can be updated, A/B tested, or rolled back independently.

Separate models better when: The two tasks have different data sources, update cadences, or precision/recall requirements.

One model acceptable when: Data is scarce and the tasks are highly correlated; training and

5

Mind vs. Data

5 The AI Hierarchy of Needs

Monica Rogati's pyramid

- The data science hierarchy of needs (2017 essay, widely cited)
- AI/ML sits at the *top* of the pyramid
- But it can only exist if the lower layers are solid



The practical consequence

"Without a data foundation, there is no data science."

The great debate: mind vs. data

Peter Norvig (Google):
"We don't have better algorithms. We just have more data."

Rich Sutton — The Bitter Lesson (2019):
"The biggest lesson from 70 years of AI research is that general methods that leverage computation are ultimately the most effective."

Judea Pearl:
"Data is profoundly dumb."

5 Data Scale Matters: The GPT Example

The scaling story

- GPT-2 (2019): trained on **10 billion tokens**
- GPT-3 (2020): trained on **500 billion tokens**
- 50× more data, same general architecture
- Result: qualitatively different capabilities emerged
- Not from a fundamentally better algorithm — from more data and compute

The practical lesson

In many domains, collecting and cleaning more high-quality data beats spending weeks on model architecture improvements.

What this means for your projects

- **Before tuning**: ask whether you need more data
- **Data quality** often matters more than data quantity
- The data collection pyramid (Rogati) is not glamorous — but it is what enables everything above it
- "Algorithmic improvements are what win headlines. Data improvements are what win production."

For this course

We will study both sides: when to invest in better algorithms, when to invest in better data. Neither answer is always right.

5 What We Covered Today

§1 From ML Metrics to Business Metrics

- Why ML projects get killed
- Business metric → ML metric translation
- Netflix take-rate worked example
- Deployment maturity curve

§2 Four Requirements for ML Systems

- Reliability: silent vs. loud failures
- Scalability: 8K models, Prime Day
- Maintainability and Adaptability

§3 The Iterative Development Process

- The 6-step cycle (never-ending)
- What each step involves
- Why cycles go backwards

§4–5 Problem Framing & Mind vs. Data

- Inputs, outputs, objective function
- Task types: regression, binary, multiclass, multilabel
- Framing matters: app prediction example
- Decoupling objectives: newsfeed ranking
- Data foundation is non-negotiable

Thank You!

Frame the problem right. Everything else follows.

Priyansh Saxena

All materials at:

https://www.priyanshsaxena.com/sst-ml-system-design

